



BSc (Hons) in Information Technology
IT4100 - SQA
Year 4 Semester 2 - 2026

Assignment 2 – Final Report

Group Member	IT Number
De Silva Y H E	IT22926180
Ganepola G A S S	IT22321022
Wijesuriya M W A D	IT22369574
Vidyasarani G G T	IT22886422

1. Introduction

1.1 Overview of the System Tested

PixelsSuite (<https://www.pixelssuite.com/>) is a free, browser-based image processing toolkit that allows users to manipulate digital images directly in their browser without installing any software. All processing is performed client-side, ensuring uploaded images are not transmitted to external servers.

1.2 Purpose and Scope of Testing

The purpose of this engagement is to evaluate the functional correctness, reliability, and usability of the PixelsSuite platform. Testing covers four feature modules assigned to each group member: Image to PDF Converter, Image Converter (PNG/JPG/WEBP), Image Compress, and Image Crop. Scope includes functional validation, UI behavior, edge case handling, and automated regression.

1.3 Test Plan

Objectives

- Verifying each feature produces correct and expected output for valid inputs
- Confirm the UI provides appropriate feedback for both valid and invalid interactions
- Identify functional defects, UI inconsistencies, and edge-case failures
- Automate key regression scenarios using each member's assigned tool
- Document all findings with reproducible evidence and screenshots

Testing Methodologies

Methodology	Description
Manual Testing	Structured test cases executed by each team member covering functional, UI, and edge-case scenarios. Results documented with pass/fail status and screenshots.
Automated Testing	Script-based test execution using browser automation frameworks. Each member used a separate tool to cover the system/website.

Tools and Technologies

Feature / Member	Tool	Feature Module
De Silva Y H E	Selenium	Image Compress
Ganepola G A S S	TestNG	Document Converter
Wijesuriya M W A D	Playwright	Image Converter (PNG/JPG/WEBP)
Vidyasarani G G T	Cypress	Image Crop

2. Manual Testing Overview

2.1 Approach

Manual testing was performed by all four team members on their respective feature modules. Each tester followed predefined test case scripts and additionally performed exploratory testing to identify unexpected behaviour. Testing covered three categories: Functional Testing, UI/UX Testing, and Edge Case Testing.

2.2 Manual Test Results Summary

Feature / Member	Total	Passed	Failed	Pass Rate
De Silva Y H E (Image Compress)	9	6	3	66.7%
Ganepola G A S S (Document Converter)	7	6	1	85.71%
Wijesuriya M W A D (Image Converter)	7	5	2	71.42%
Vidyasarani G G T (Image Crop)	7	5	2	71.42%

2.3 Defects Found – Manual Testing

ID	Title	TC Ref	Severity	Priority	Module	Description
DEF-001	File size limit not enforced	TC-CI-001	Medium	Medium	Crop Image	Large Files Allowed
DEF-002	Not Handle Unsupported format	TC-CI-002	Medium	Medium	Crop Image	No error display when upload unsupported format image
DEF-003	Output Format Dropdown Not Populated After Image Upload	TC-IC-004	High	High	Image Converter	After uploading a valid image, the output format dropdown does not display any options and remains in a disabled state, preventing the user from selecting a conversion format

DEF-004	Clear Action Does Not Reset Preview Image Properly	TC-IC-005	Medium	Medium	Image Converter	After clicking Clear, Preview card retains the previously uploaded image instead of fully resetting to empty state
DEF-005	Upload Corrupted Image File	TC-IP-007	High	High	Open	No upfront validation for corrupted image files
DEF-006	Upload non-image file(.txt)	TC-IC-007	Medium	High	Compress Image	Uploading an unsupported file (e.g., .txt) does not show a proper error message or validation
DEF-007	Upload large image (>10MB)	TC-IC-008	High	High	Compress Image	System becomes slow or unresponsive when uploading large files
DEF-008	Compress without upload	TC-IC-009	Medium	High	Compress Image	No warning message displayed when user clicks Compress without uploading a file

3. Automated Testing Overview

3.1 Tool Selection Rationale

Member	Tool	Reasoning
De Silva Y H E	Selenium	Widely used industry-standard tool with strong community support. Suitable for cross-browser testing and integrates well with many frameworks and languages like Java.
Ganepola G A S S	TestNg	Provides advanced testing features such as annotations, parallel execution, and detailed reporting. Works well with Selenium for structured test management.
Wijesuriya M W A D	Playwright	Modern automation tool with fast execution, built-in auto-waiting, and excellent cross-browser support (Chromium, Firefox, WebKit). Ideal for reliable and stable UI testing.
Vidyardarani G T	Cypress	Easy to set up and use with real-time reloading and debugging. Best suited for frontend testing with fast execution and clear test visualization.

3.2 Automated Testing by Member Summary

Feature / Member	Total	Passed	Failed	Pass Rate
De Silva Y H E	27	17	10	62.96%
Ganepola G A S S	30	23	7	76.66%
Wijesuriya M W A D	66	62	4	93.93%
Vidyardarani G T	23	21	2	91.3%

3.3 Defects Found – Automated Testing

ID	Title	TC Ref	Severity	Priority	Module	Description
DEF-001	Silent file-type rejection – no error message shown	RS_002	Medium	Medium	Resize Image	Multiple (Resize, Crop, Compress, Convert, Tools)
DEF-002	Download output not verified – corrupt/blank files undetected	DC_001, DC_002, DC_003	High	High	Document Converter	Test suite never clicks the Download button nor checks the output file size or content. A tool could produce a corrupt or zero-byte file and all three Document Converter tests would still pass.
DEF-003	Meme text not verified on canvas – text rendering failure undetected	MT_003	High	High	Meme Generator	MT_003 fills Top/Bottom text fields and checks only for a "Download Meme" or "Preview" element. The test never inspects the canvas to confirm the typed text is actually rendered on the meme image.
DEF-004	Over-permissive assertions – any page text triggers pass	RS_001, CR_001–003, CP_001–003, IC_001–003, MT_001–002	Medium	High	Multiple (Resize, Crop, Compress, Convert, Tools)	expectOneOfTexts() accepts broad patterns such as /Preview/i or /Download/i. A broken tool page that still contains a navigation link containing "Preview" satisfies the assertion, hiding functional regressions.
DEF-005	Convert to JPG incorrectly accepts JPG file	TC-001	Medium	High	Image Converter – Convert to JPG	When a user uploads any JPG file to the convert to JPG feature, the system goes ahead and accepts the file and works on it without giving a warning or a validation failure. The converter should discard the input file or inform the user that the file is

						already in JPG format and therefore, does not need conversion. This means that there is no input validation logic in the module.
DEF-006	Uploaded image remains after changing converter format	TC-002	Low	Medium	Image Converter – Format Switch	In one converter format (e.g. Convert to PNG) the user uploads a picture, and in another converter format (e.g. Convert to JPG), the picture will still be visible in the new converter section. When changing converter types, the system should also clear the uploaded file because it can be confusing and can result in unwanted conversions.
DEF-007	No error validation (uploads)	IP-008	High	High	Image to pdf	When you add a corrupted file, No upfront validation for corrupted image files

5. Conclusion

5.1 Summary of Findings

This was tested on Pixels Suite web application (<https://www.pixelssuite.com/>), which includes functional testing, UI testing and edge case testing on various features with the exception of the Transliteration feature. Manual and automated testing has been conducted with manual testing detecting usability and functional flaws based on structured test cases and automated testing confirming core functionality-based testing being faster and more repeatable. The number of 131 test cases was designed and run, with 116 passing and 15 failing. Playwright, Selenium, Cypress, and TestNG were used to conduct automated testing, with each of the team members responsible for one of the tools. There were a number of functional bugs that were found during both tests, such as input validation misbehaviors, odd UI reactions, and erroneous output in some features. Most of the defects were medium to high-severity, and the general impression is that the application has a minimum of functionality but could use improvement in input processing and error reporting, and cross-browsing compatibility.

5.2 Comparison of Automation Tools

Criterion	Playwright	Cypress	Selenium	TestNG
Language support	JS, TS, Python, Java, C#	JS, TS only	Java, Python, C#, Ruby, JS	Java only
Browser support	Chromium, Firefox, WebKit	Chrome, Firefox, Edge	All major browsers	None (pairs with Selenium)
Setup complexity	Low	Low	Low	Low
Parallel execution	Built-in, free	Paid (Cypress Cloud)	Via Selenium Grid	Built-in via XML

5.3 Challenges Faced

- At the beginning tool setup and configuration was difficult.
- Client-side processing meant that it was hard to claim the contents of output files, and that tests needed to be based on download triggers and UI state change as proxy success.
- The non-functional format dropdown inhibited some of the planned automated test scenarios of the Image Converter module.
- Large automation suites demand proper folder and structure.

5.4 Improvements Suggested

1. Improvements Suggested

Continuous Integration (CI) is another tool to be implemented to facilitate Automated Testing. - A CI tool like GitHub Actions or Jenkins could be integrated to run tests automatically on each code change and minimize the number of people needed, as well as accelerate the detection of defects.

2. Increase Test coverage with performance and load testing.

Performance testing tools such as Apache JMeter or k6 would be incorporated to expose the way the application can withstand heavy traffic and multiple simultaneous users and identify bottlenecks that cannot be identified by functional testing.

3. Establish a Standardized Defect Tracking System.

A specific bug tracking system like Jira or Bugzilla would help to consolidate the defects management process with the appropriate tagging of priorities and severity, enhance the traceability and collaboration among the teams.

4. Add Cross-Browser and Cross-Device Testing. - It would be important to extend the scope of test execution to various browsers (Chrome, Firefox, Safari, Edge) and screen sizes with the help of such tools as BrowserStack to make sure that the application provides the same experience in terms of UI/UX across all platforms and devices.

References

1. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.
2. Selenium Documentation. (2024). *Selenium WebDriver*. Retrieved from <https://www.selenium.dev/documentation/>
3. Playwright Documentation. (2024). *Playwright for Python/JavaScript*. Retrieved from <https://playwright.dev/docs/intro>
4. Cypress Documentation. (2024). *Why Cypress?* Retrieved from <https://docs.cypress.io/guides/overview/why-cypress>
5. TestNG Documentation. (2024). *TestNG - Introduction*. Retrieved from <https://testng.org/doc/>
6. ISTQB. (2023). *Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board. Retrieved from <https://www.istqb.org>
7. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Appendix

Appendix A – Test Coverage Matrix

Feature Module	Member	Manual TCs	Auto TCs	Defects	Tool
Image Compress	De Silva Y H E	09	27	03	Selenium
PDF Converter	Ganepola G A S S	07	30	01	TestNG
Image Converter	Wijesuriya M W A D	07	21	04	Playwright
Image Crop	Vidyasarani G G T	07	23	04	Cypress